

Propveil: Research Synthesis & Psychological Product Strategy

Human Behavior, Robert Greene's Principles, and Freemium Conversion

Date: April 2026 | **Author:** Manus AI

"When you show yourself to the world and display your talents, you naturally stir all kinds of resentment, envy, and other manifestations of insecurity... you cannot spend your life worrying about the petty feelings of others." — Robert Greene, *The 48 Laws of Power*

This document synthesizes deep research into human psychology, behavioral economics, and the complete works of Robert Greene, applying these insights directly to the product design and monetization strategy of **Propveil**, a Flutter-based rent management application for the Cameroonian market.

1. Robert Greene's Principles Applied to Propveil

Robert Greene's works explore the underlying mechanics of power, influence, and human nature. Propveil is designed not just as a utility, but as an instrument of control and mastery for the Cameroonian landlord.

1.1 The 48 Laws of Power (Strategic Control)

- **Law 3: Conceal your intentions.** The app's UI should feel simple ("just a rent tracker") while slowly revealing its true power (AI triage, automated late fees, OHADA compliance) as the user engages deeper.
- **Law 11: Learn to keep people dependent on you.** This is the core retention metric. As landlords input tenant histories, MoMo payment records, and lease documents, the switching cost becomes prohibitive. Propveil becomes the single source of truth.
- **Law 27: Play on people's need to believe.** Propveil is positioned as the definitive tool for the modern, professional property manager, elevating them above the chaotic, informal cash-collection methods of the past ¹.

1.2 The Art of Seduction (User Engagement)

- **The Coquette (Hot and Cold):** The freemium model must act as the Coquette. It provides immense value (hot) but gates the most powerful features (cold) right when

the user desires them most, creating an irresistible pull to upgrade.

- **The Charmer:** The Anthropic Claude-powered AI assistant acts as the Charmer, adapting to the user's language (French/English) and offering personalized insights based on their specific portfolio performance.

1.3 The Laws of Human Nature (Behavioral Design)

- **The Law of Irrationality:** Users make emotional decisions and rationalize them later ². The app must first alleviate the *emotional stress* of rent collection before selling the *logical benefit* of analytics.
- **The Law of Role-playing:** Users want to feel like "professional property managers." Propveil's branding, unbranded PDF receipts, and automated WhatsApp reminders allow them to play this elevated role.
- **The Law of Shortsightedness:** Landlords often focus on immediate cash rather than long-term yield. The dashboard must visually contrast short-term gains with long-term portfolio valuation to motivate the use of Pro features.

1.4 Mastery (User Journey)

- **The Apprenticeship Phase:** The Free Tier is the apprenticeship. It teaches the landlord the basics of digital rent collection and tenant tracking.
 - **The Mastery Phase:** The Pro Tier provides the tools for true mastery—automated late fees, AI maintenance triage, and OHADA-compliant legal protection.
-

2. Behavioral Science & Product Psychology

2.1 The Hooked Model (Nir Eyal)

Propveil utilizes the Hooked Model to manufacture habitual use ³:

- **Trigger:** Automated, localized WhatsApp notifications (e.g., "Rent due in 3 days for Unit 4").
- **Action:** The landlord opens Propveil to view the dashboard.
- **Variable Reward:** Seeing early payments via NotchPay; receiving AI insights on portfolio yield; clearing maintenance requests.
- **Investment:** Adding properties, customizing legal templates, and building tenant credit histories.

2.2 Cognitive Biases in App Design

- **Loss Aversion:** People feel the pain of losses twice as strongly as the pleasure of gains 4 . Propveil frames upgrades around loss: *"You are missing XAF 45,000 in uncollected late fees this month. Upgrade to Pro to automate collection."*
- **The Endowment Effect:** Users value what they already have. Once a landlord sets up their dashboard and sees their properties visualized, they will pay to avoid losing that organized state.
- **Social Proof:** The app utilizes localized social proof: *"Join 1,200 landlords in Douala using Propveil."*

3. Freemium Monetization Strategy

The monetization strategy is engineered using the **Decoy Effect** and **Price Anchoring** to drive users toward the Pro tier 5 .

3.1 The Psychology of the Free Tier

The "Zero-Price Effect" removes all friction for adoption 4 . However, the free tier must create natural friction at growth moments. We never gate the core value (rent collection); we gate the *scale* and *automation* of that value.

3.2 Tier Structure & Upgrade Triggers

Feature	Gérant Starter (Free)	Gérant Pro (XAF 15,000/mo)	Agence Pro (XAF 45,000/mo)
Properties	Up to 3	Unlimited	Unlimited
Rent Collection	Manual WhatsApp triggers	Automated WhatsApp schedules	Automated + Bulk Messaging
Payments	NotchPay (MoMo/Orange)	NotchPay + Instant Payouts	NotchPay + Custom Routing
History	3 Months	Unlimited	Unlimited
AI Triage	✗	✓ (Bilingual)	✓ (Bilingual)
Legal	Basic PDF Receipts	OHADA Lease Generation	Custom Lease Engine
Branding	Propveil Watermark	Unbranded	Custom Agency White-Label

3.3 Psychological Upgrade Moments

Upgrades are prompted not randomly, but at the exact moment of psychological need:

1. **The Scaling Trigger:** When the user attempts to add a 4th property. (*Desire for expansion*)
2. **The Dispute Trigger:** When a tenant disputes a payment or maintenance issue. The app prompts: *"Upgrade to Pro for OHADA-compliant lease protection and AI-logged maintenance trails."* (*Fear of loss/legal trouble*)
3. **The Fatigue Trigger:** When the user manually sends the 10th WhatsApp reminder. The app prompts: *"Tired of texting? Upgrade to Pro to automate all rent reminders."* (*Desire for ease/mastery*)

References

- [1] Graham Mann, "48 Laws of Power: Complete List & Summary,"
- [2] Graham Mann, "Laws of Human Nature: All 18 Laws Explained,"
- [3] Nir Eyal, "The Hooked Model: How to Manufacture Desire in 4 Steps,"
- [4] Phoenix Strategy Group, "Freemium vs. Subscription: Behavioral Insights,"
- [5] Fungies.io, "SaaS Pricing Psychology: The Complete Guide,"

Propveil: Flutter Architecture & Technical Plan

Cameroonian Rent Management Mobile App

Date: April 2026 | **Author:** Manus AI

This document details the technical architecture, database schema, API integrations, and development roadmap for **Propveil**, built exclusively with the Flutter framework for mobile (Android/iOS) and web.

1. System Architecture (Flutter-First)

Propveil utilizes a modern, serverless multi-tenant architecture, prioritizing a robust, offline-capable mobile experience built in Flutter.

1.1 The Flutter Frontend

The mobile application is built entirely in **Flutter** (not React Native) using Clean Architecture principles.

- **State Management:** `riverpod` (for predictable, testable state management and dependency injection).
- **Routing:** `go_router` (for deep linking and nested navigation, essential for handling WhatsApp redirect links).
- **Local Storage (Offline Mode):** `isar` database. Crucial for Cameroon's variable internet connectivity. Managers can view properties, tenants, and pending tasks offline; actions sync when the connection is restored.
- **Localization:** `flutter_localizations` and `intl` for seamless switching between French and English (bilingual support).
- **UI/UX Framework:** Custom Material 3 design system, utilizing the psychological principles of the "Charmer" archetype (clean, effortless, intuitive).

1.2 The Backend (Serverless & Agentic)

- **API Layer:** Node.js (NestJS) deployed on Google Cloud Run. Chosen for its strict TypeScript architecture, making it highly reliable for financial transactions.
- **Development Workflow:** Built using Claude Code (Anthropic's CLI agent) within the Google Antigravity IDE for rapid iteration and serverless deployment.
- **Database:** Google Cloud SQL (PostgreSQL) implementing Row-Level Security (RLS) to ensure strict data isolation between different property managers (tenants).

2. Flutter Folder Structure (Clean Architecture)

The Flutter project is structured to separate concerns, making the app scalable and testable.

Plain Text

```
lib/
├── core/                # Shared utilities, theme, localization, constants
│   ├── network/        # Dio interceptors, API clients
│   ├── theme/          # Propveil colors, typography, Material 3 config
│   ├── errors/         # Failure models, exception handling
│   └── utils/          # Formatters (XAF currency, dates)
├── data/               # Data layer (Repositories, Data Sources)
│   ├── models/         # DTOs (Data Transfer Objects) mapping to JSON
│   ├── datasources/    # Remote (NestJS API) and Local (Isar DB) sources
│   └── repositories/   # Implementation of domain repository interfaces
```

└─ domain/	# Business logic (Entities, Use Cases)
└─ └─ entities/	# Pure Dart classes (Property, Tenant, Transaction)
└─ └─ repositories/	# Abstract interfaces for data layer
└─ └─ usecases/	# Specific actions (e.g., CollectRent, TriageMaintenance)
└─ presentation/	# UI layer (Widgets, Screens, State)
└─ └─ providers/	# Riverpod state providers
└─ └─ screens/	# Main UI views (Dashboard, PropertyList, TenantDetails)
└─ └─ widgets/	# Reusable UI components (PropertyCard, NotchPayButton)
└─ main.dart	# Entry point, ProviderScope, GoRouter setup

3. Database Schema (PostgreSQL with RLS)

The schema is designed for multi-tenancy. Every table enforces Row-Level Security based on the `manager_id`.

managers (App Subscribers)

SQL

```
CREATE TABLE managers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  full_name VARCHAR(255) NOT NULL,
  phone_number VARCHAR(20) UNIQUE NOT NULL, -- Primary MoMo/Orange Money number
  subscription_tier VARCHAR(50) DEFAULT 'STARTER', -- STARTER, PRO, ENTERPRISE
  notchpay_customer_id VARCHAR(100),
  role VARCHAR(20) DEFAULT 'OWNER',
  trust_score DECIMAL(3, 2) DEFAULT 5.00,
  settings_json JSONB,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

properties & units

SQL

```
CREATE TABLE properties (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  manager_id UUID REFERENCES managers(id) ON DELETE CASCADE,
  name VARCHAR(255) NOT NULL,
  location_city VARCHAR(100) NOT NULL,
  address_line TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE units (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```

property_id UUID REFERENCES properties(id) ON DELETE CASCADE,
unit_number VARCHAR(50) NOT NULL,
rent_amount_xaf DECIMAL(15, 2) NOT NULL,
is_occupied BOOLEAN DEFAULT FALSE,
meter_reading_last DECIMAL(10, 2),
inventory_json JSONB
);

```

tenants & leases

SQL

```

CREATE TABLE tenants (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  unit_id UUID REFERENCES units(id),
  full_name VARCHAR(255) NOT NULL,
  momo_number VARCHAR(20) NOT NULL,
  whatsapp_consent BOOLEAN DEFAULT TRUE,
  credit_score_internal INTEGER DEFAULT 700,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE leases (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID REFERENCES tenants(id),
  start_date DATE NOT NULL,
  end_date DATE,
  legal_text_fr TEXT, -- OHADA-compliant generated text
  signed_at TIMESTAMPTZ
);

```

transactions (Rent & Payouts)

SQL

```

CREATE TABLE transactions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  manager_id UUID REFERENCES managers(id),
  tenant_id UUID REFERENCES tenants(id),
  amount_xaf DECIMAL(15, 2) NOT NULL,
  type VARCHAR(50) NOT NULL, -- RENT, SUBSCRIPTION, PAYOUT, LATE_FEE
  status VARCHAR(50) DEFAULT 'PENDING',
  notchpay_ref VARCHAR(100) UNIQUE,
  payout_status VARCHAR(50) DEFAULT 'PENDING',
  late_fee_applied BOOLEAN DEFAULT FALSE,

```

```
created_at TIMESTAMPTZ DEFAULT NOW()
);
```

4. API Integrations

Propveil relies on four critical external APIs to deliver its value proposition.

- NotchPay API (Payment Gateway):** Handles all financial transactions.
 - Payments API:* Processes rent from tenants via MTN Mobile Money and Orange Money.
 - Transfers API:* Triggers instant payouts to the landlord's MoMo wallet upon rent collection.
 - Webhooks:* Listens for payment confirmations to update the Flutter UI and trigger WhatsApp receipts.
- Anthropic Claude API (AI Triage):** Powers the bilingual (FR/EN) maintenance assistant. Tenants report issues; Claude diagnoses the problem, provides immediate troubleshooting steps, and logs a structured work order for the landlord.
- WhatsApp Business API:** The primary communication engine. Automates rent reminders, sends digital receipts, and updates tenants on maintenance status without requiring them to download a separate app.
- Google Maps API:** Used within the Flutter app for property location tracking and neighborhood visualization.

5. Development Roadmap (18 Weeks)

The project will be executed using Claude Code for accelerated backend development and Flutter for the frontend.

Phase 1: Foundation & Architecture (Weeks 1-4)

- Initialize NestJS backend and PostgreSQL database on Google Cloud.
- Set up Flutter project with Clean Architecture, Riverpod, and Isar.
- Implement Row-Level Security (RLS) for multi-tenancy.
- Design the core UI components (Material 3, Propveil branding).

Phase 2: Core Management & Payments (Weeks 5-9)

- Build Property, Unit, and Tenant CRUD operations in Flutter.
- Integrate NotchPay API for MTN/Orange Money rent collection.

- Implement webhook listeners for real-time transaction status updates.
- Develop the offline sync mechanism using Isar.

Phase 3: Automation & AI Triage (Weeks 10-14)

- Integrate WhatsApp Business API for automated reminders and receipts.
- Build the Anthropic Claude integration for the bilingual AI Maintenance Triage.
- Implement the automated late fee calculation engine (Cron jobs).
- Develop the OHADA-compliant lease generation module.

Phase 4: Analytics, Freemium Tiers & Launch (Weeks 15-18)

- Build the advanced financial analytics dashboard (NOI, cash flow).
- Implement the psychological upgrade triggers (Decoy Effect, Feature Gating) for the Gérant Starter, Pro, and Agence Pro tiers.
- Final bilingual (FR/EN) QA testing and localization review.
- Deploy backend to Google Cloud Run; publish Flutter app to Google Play Store and Apple App Store.